

nanoGPT ile Transformer'ı Sıfırdan Anlamak

Bir GitHub repo'sundan başlayıp kendi GPT modelimizi eğittiğimiz yolculuğun çalışma notu.

İçindekiler

- [1. Yolculuğa Başlangıç](#)
- [2. Transformer Nedir? Temel Kavramlar](#)
- [3. nanoGPT'nin Yapısı](#)
- [4. LayerNorm — Isınma Turu](#)
- [5. CausalSelfAttention — Transformer'ın Kalbi](#)
- [6. MLP — Düşünme Katmanı](#)
- [7. Block — Bir Transformer Katı](#)
- [8. GPT — Tüm Model](#)
- [9. Generate — Metin Üretme](#)
- [10. Pratik: Shakespeare Modeli Eğitme](#)
- [11. Sonuçlar ve Değerlendirme](#)

Yolculuğa Başlangıç

Başlangıçta `meta-llama/llama` repo'sunu inceledik ama bu repo deprecated bir Llama 2 inference kodu. Transformer mimarisini **gerçekten** anlamak için Karpathy'nin `nanoGPT` repo'suna yöneldik. Sebebi: kod sade, ~300 satır, eğitim odaklı.

Hedef: Bir LLM'in iç yapısını, kodun her satırıyla beraber anlamak.

Transformer Nedir? Temel Kavramlar

Dil Modeli ne yapar?

Bir dil modeli tek görev yapar: **bir sonraki token'ı tahmin etmek**. ChatGPT, Llama, Claude — hepsi sadece "bu kelimelerden sonra ne gelmeli?" sorusunu defalarca cevaplayan makineler. Bu döngüye **autoregressive generation** denir.

Token

Model "kelime" görmez. Metin önce **token**'lara bölünür (yaklaşık kelime parçası):

```
"Merhaba dünya" → ["Mer", "haba", "dün", "ya"] → [1523, 8821, 445, 92]
```

Her token'a bir sayı atanır. Model **sayı dizileri** ile çalışır.

Embedding

Sayılar tek başına anlamlı değil. Her token bir **vektöre** dönüştürülür (örn. 768 boyutlu):

```
1523 → [0.23, -0.81, 0.05, ..., 0.44]
```

Benzer anlamlı kelimelerin vektörleri yakın olur ("kral" ve "kraliçe"). Model bunu eğitim sırasında kendi öğrenir.

Attention — En önemli kavram

"Banka kenarında oturdum, su akıyordu."

Buradaki "banka" finansal kurum mu, nehir kenarı mı? "Su akıyordu" sayesinde anlıyoruz. Bir kelimenin anlamı diğer kelimelere bakarak netleşir.

Attention her token'ın diğer token'lara "siz benim için ne kadar önemlisiniz?" diye sormasıdır. Üç vektör üretilir:

- **Query (Q)**: "Ben neyi arıyorum?"
- **Key (K)**: "Bende ne bilgi var?"
- **Value (V)**: "Benden bilgi alacaksan, şunu al"

Her token'ın Q'su diğer token'ların K'larıyla karşılaştırılır. Yüksek skor = ilgili. Sonra V'ler ağırlıklı toplanır.

Multi-head

Bir cümlede birden fazla ilişki türü var: özne-yüklem, zaman, neden-sonuç... Bu yüzden attention birden fazla **kafa (head)** ile paralel yapılır. Her kafa farklı bir ilişki türü öğrenir.

Llama'ya özgü detaylar

- **RMSNorm**: Normalizasyon yöntemi (LayerNorm'un sade hali)
- **RoPE (Rotary Position Embedding)**: Pozisyon bilgisini Q ve K'yı döndürerek verir
- **KV Cache**: Generation hızı için K ve V değerlerini saklar

nanoGPT'nin Yapısı

İki önemli dosya:

Dosya	Ne işe yarar
<code>model.py</code>	Tüm transformer mimarisi. ~300 satır.
<code>train.py</code>	Eğitim döngüsü
<code>sample.py</code>	Eğitilmiş modelden metin üretme

`model.py` 'deki 5 ana sınıf (basitten karmaşığa):

1. `LayerNorm`  normalizasyon
2. `CausalSelfAttention`  attention (transformer'ın kalbi)
3. `MLP`  feedforward katman
4. `Block`  bir transformer bloğu
5. `GPT`  tüm model

LayerNorm — Isınma Turu

PyTorch'un temel mantığı: `nn.Module`

PyTorch'ta her bileşen `nn.Module` 'dan türer:

```
class BirseyAdli(nn.Module):
    def __init__(self):
        super().__init__()
        # Öğrenilecek ağırlıkları tanımla

    def forward(self, x):
        # Hesaplama
        return sonuc
```

- `__init__`: Ağırlıkları tanımlar
- `forward`: Veriyi işler (PyTorch otomatik çağırır)

nn.Parameter

`nn.Parameter(torch.ones(ndim))` → "Bu sayı sabit değil, eğitim sırasında değişecek, backprop güncellesin" demek.

LayerNorm kodu

```
class LayerNorm(nn.Module):
    """ LayerNorm but with an optional bias. PyTorch doesn't support simply bias=False """

    def __init__(self, ndim, bias):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(ndim))
        self.bias = nn.Parameter(torch.zeros(ndim)) if bias else None

    def forward(self, input):
        return F.layer_norm(input, self.weight.shape, self.weight, self.bias, 1e-5)
```

Ne yapıyor: Her vektörü standart bir büyüklüğe getiriyor. Matematik:

1. Ortalamayı hesapla: $\mu = (x_1 + x_2 + \dots + x_n) / n$
2. Std hesapla: $\sigma = \dots$
3. Normalize et: $(x_i - \mu) / \sigma$
4. Sonra: `weight * normalize_edilmis + bias`

Karpathy bunu yazma sebebi: PyTorch'un kendi `nn.LayerNorm`'u `bias=False` desteklemiyor.

CausalSelfAttention — Transformer'ın Kalbi

Temel kavram

$B=4$, $T=8$, $C=768$, $n_head=12$ örneğiyle gideceğiz: - 4 cümle paralel, her cümle 8 token, her token 768 boyutlu vektör - 12 attention başlığı, her biri $768/12 = 64$ boyutlu

Causal mask

GPT, **bir sonraki kelimeyi** tahmin ediyor. Eğitim sırasında "Kedi süt içti" cümlesinde "içti"yi tahmin ederken, model "içti"yi göremezse mantıklı bir görev olur. Bu yüzden **her token sadece kendisinden öncekilere bakabilmeli**.

Aşağı üçgen mask:

	t0	t1	t2	t3
t0	[✓ -inf -inf -inf]			
t1	[✓ ✓ -inf -inf]			
t2	[✓ ✓ ✓ -inf]			
t3	[✓ ✓ ✓ ✓]			

init kodu

```
class CausalSelfAttention(nn.Module):

    def __init__(self, config):
        super().__init__()
        assert config.n_embd % config.n_head == 0
        # key, query, value projections for all heads, but in a batch
        self.c_attn = nn.Linear(config.n_embd, 3 * config.n_embd, bias=config.bias)
        # output projection
        self.c_proj = nn.Linear(config.n_embd, config.n_embd, bias=config.bias)
        # regularization
        self.attn_dropout = nn.Dropout(config.dropout)
        self.resid_dropout = nn.Dropout(config.dropout)
        self.n_head = config.n_head
        self.n_embd = config.n_embd
        self.dropout = config.dropout
        # flash attention make GPU go brrrrr but support is only in PyTorch >= 2.0
        self.flash = hasattr(torch.nn.functional, 'scaled_dot_product_attention')
        if not self.flash:
            print("WARNING: using slow attention. Flash Attention requires PyTorch >= 2.0")
            # causal mask to ensure that attention is only applied to the left in the input sequence
            self.register_buffer("bias", torch.tril(torch.ones(config.block_size, config.block_size)
                                                    .view(1, 1, config.block_size, config.block_size)))
```

Önemli detay: `c_attn = Linear(768, 3*768)` → Q, K, V için **üç ayrı matris yerine tek büyük matris**. GPU'da çok daha hızlı.

Forward — adım adım

```
def forward(self, x):
    B, T, C = x.size()

    # Q, K, V'yi üret
    q, k, v = self.c_attn(x).split(self.n_embd, dim=2)
    k = k.view(B, T, self.n_head, C // self.n_head).transpose(1, 2)
    q = q.view(B, T, self.n_head, C // self.n_head).transpose(1, 2)
    v = v.view(B, T, self.n_head, C // self.n_head).transpose(1, 2)

    # Manuel attention (Flash yoksa)
    att = (q @ k.transpose(-2, -1)) * (1.0 / math.sqrt(k.size(-1)))
    att = att.masked_fill(self.bias[:, :, :T, :T] == 0, float('-inf'))
    att = F.softmax(att, dim=-1)
    att = self.attn_dropout(att)
    y = att @ v

    # Başlıkları birleştir
    y = y.transpose(1, 2).contiguous().view(B, T, C)
    y = self.resid_dropout(self.c_proj(y))
    return y
```

Boyut akışı

```
x: (B, T, C)                [4, 8, 768]
|
├─ c_attn      → (B, T, 3C)   [4, 8, 2304]
├─ split       → q,k,v: (B, T, C) her biri
├─ view+transpose → (B, nh, T, hs)   [4, 12, 8, 64]
|
├─ q @ k^T / √hs → att: (B, nh, T, T) [4, 12, 8, 8]   ham skorlar
├─ causal mask  → gelecek -inf
├─ softmax      → olasılık dağılımı
├─ dropout
├─ att @ v      → y: (B, nh, T, hs)
|
├─ transpose+view → y: (B, T, C)           başlıkları birleştir
├─ c_proj       → y: (B, T, C)
├─ dropout
```

Önemli kavramlar: - $\sqrt{d}$ 'ye bölmek (scaled dot-product): büyük skorların softmax'ı keskinleştirmesini engeller - Softmax: skorları olasılığa çevirir, $-\infty \rightarrow 0$ olur (mask çalışır) - Output boyutu = input boyutu $\rightarrow$ bloklar üst üste istiflenebilir

MLP — Düşünme Katmanı

Kavram

Attention "token'lar arası iletişim", MLP "her token kendi içinde düşünme":

- **Attention** = token'lar arası bilgi aktarımı
- **MLP** = her token bağımsız işlenir

Yapı: **Genişlet $\rightarrow$ Aktivasyon $\rightarrow$ Daralt**

768 $\rightarrow$ 3072 $\rightarrow$ 3072 $\rightarrow$ 768

GELU aktivasyon

ReLU'nun yumuşatılmış versiyonu. Non-linearity ekler. GPT-2, BERT, hepsi GELU kullanıyor.

MLP kodu

```
class MLP(nn.Module):

    def __init__(self, config):
        super().__init__()
        self.c_fc = nn.Linear(config.n_embd, 4 * config.n_embd, bias=config.bias)
        self.gelu = nn.GELU()
        self.c_proj = nn.Linear(4 * config.n_embd, config.n_embd, bias=config.bias)
        self.dropout = nn.Dropout(config.dropout)

    def forward(self, x):
        x = self.c_fc(x)
        x = self.gelu(x)
        x = self.c_proj(x)
        x = self.dropout(x)
        return x
```

Boyut akışı

```
Girdi:      (B, T, 768)
c_fc       → (B, T, 3072)   genişledi
gelu       → (B, T, 3072)   non-linearity
c_proj     → (B, T, 768)    daraldı
dropout    → (B, T, 768)
```

Block — Bir Transformer Katı

Residual connection

ResNet'ten gelen kritik fikir:

$$y = x + f(x)$$

Yararı: 1. Gradient'ler kolay akar (vanishing gradient sorunu çözülür) 2. En kötü durumda block hiçbir şey yapmaz → kötü bir şey yapmaz

Pre-norm vs Post-norm

GPT-2 ve sonrası **pre-norm** kullanıyor (LayerNorm önce, asıl işlem sonra):

```
x → LayerNorm → attention → +x → LayerNorm → mlp → +x → çıktı
```

Block kodu

```
class Block(nn.Module):

    def __init__(self, config):
        super().__init__()
        self.ln_1 = LayerNorm(config.n_embd, bias=config.bias)
        self.attn = CausalSelfAttention(config)
        self.ln_2 = LayerNorm(config.n_embd, bias=config.bias)
        self.mlp = MLP(config)

    def forward(self, x):
        x = x + self.attn(self.ln_1(x))
        x = x + self.mlp(self.ln_2(x))
        return x
```


13 satır. İki "alt-blok": biri attention'lı, biri MLP'li. Her ikisinin de kendi LayerNorm'u ve kendi residual bağlantısı var.

GPT — Tüm Model

Yeni kavramlar

Token Embedding: `nn.Embedding(vocab_size, n_embd)` → her token için bir vektör tablosu.

Position Embedding: Attention sıra bilgisi yok! Bu yüzden her pozisyona da bir vektör atanıp **token embedding ile toplanır**:

```
final_embedding = token_embedding + position_embedding
```

Weight tying: Embedding tablosu ile lm_head aynı ağırlıkları paylaşır. ~38M parametre tasarrufu.

GPTConfig

```
@dataclass
class GPTConfig:
    block_size: int = 1024
    vocab_size: int = 50304    # GPT-2 vocab, 64'ün katına yuvarlanmış
    n_layer: int = 12
    n_head: int = 12
    n_embd: int = 768
    dropout: float = 0.0
    bias: bool = True
```

Bu **GPT-2 small**'in ayarları (124M parametre).

GPT init (önemli kısım)

```
class GPT(nn.Module):

    def __init__(self, config):
        super().__init__()
        self.config = config

        self.transformer = nn.ModuleDict(dict(
            wte = nn.Embedding(config.vocab_size, config.n_embd),
            wpe = nn.Embedding(config.block_size, config.n_embd),
            drop = nn.Dropout(config.dropout),
            h = nn.ModuleList([Block(config) for _ in range(config.n_layer)]),
            ln_f = LayerNorm(config.n_embd, bias=config.bias),
        ))
        self.lm_head = nn.Linear(config.n_embd, config.vocab_size, bias=False)
        self.transformer.wte.weight = self.lm_head.weight  # weight tying
```

Bileşen	Ne
wte	Token embedding tablosu (50304, 768)
wpe	Pozisyon embedding tablosu (1024, 768)
h	12 transformer block listesi
ln_f	Son LayerNorm
lm_head	Vektörü 50304 token skoruna çeviren matris

GPT forward

```
def forward(self, idx, targets=None):
    device = idx.device
    b, t = idx.size()
    pos = torch.arange(0, t, dtype=torch.long, device=device)

    # forward the GPT model itself
    tok_emb = self.transformer.wte(idx)          # (b, t, n_embd)
    pos_emb = self.transformer.wpe(pos)          # (t, n_embd)
    x = self.transformer.drop(tok_emb + pos_emb)
    for block in self.transformer.h:
        x = block(x)
    x = self.transformer.ln_f(x)

    if targets is not None:
        # eğitim: tüm pozisyonlar için tahmin + loss
        logits = self.lm_head(x)
        loss = F.cross_entropy(logits.view(-1, logits.size(-1)), targets.view(-1), ignore_index=-1)
    else:
        # inference: sadece son pozisyon
        logits = self.lm_head(x[:, [-1], :])
        loss = None

    return logits, loss
```

Tüm akış

```
idx: (B, T)
|
├ wte(idx)      → tok_emb: (B, T, C)      her token vektöre
├ wpe(pos)      → pos_emb: (T, C)         her pozisyon vektöre
├ tok + pos     → x: (B, T, C)            anlam + konum
├ drop          → x
|
├ Block 1..12   → x: (B, T, C)            "anla, düşün" tekrar
|
├ ln_f          → x: (B, T, C)            son normalizasyon
|
├ lm_head       → logits: (B, T, V)       sözlük skorları
|
└ cross_entropy(logits, targets) → loss
```

Cross entropy

Sınıflandırma için standart kayıp fonksiyonu. "Modelin tahmin ettiği olasılık dağılımı ile gerçek doğru cevap ne kadar farklı?"

Generate — Metin Üretme

Autoregressive döngü

```
Başlangıç: "Bir varmış"  
Tur 1: Modele "Bir varmış" ver tahmin: "bir"  
Tur 2: Modele "Bir varmış bir" ver tahmin: "yokmuş"  
Tur 3: "Bir varmış bir yokmuş" ver "."  
...
```

Temperature

Logitleri softmax'tan önce bir sayıya bölme: `logits / T`

- $T < 1$ → muhafazakar, kestirilebilir
- $T = 1$ → orijinal dağılım
- $T > 1$ → yaratıcı, sürprizli

Pratikte 0.7-1.0 arası kullanılır.

Top-k

En yüksek k token dışındakileri `-inf` yapar. Düşük olasılıklı saçma token'ları engeller. Genelde k=40 veya 200.

Generate kodu

```
def generate(self, idx, max_new_tokens, temperature=1.0, top_k=None):
    """
    Take a conditioning sequence of indices idx (LongTensor of shape (b,t)) and complete
    the sequence max_new_tokens times, feeding the predictions back into the model each time.
    """
    for _ in range(max_new_tokens):
        # if the sequence context is growing too long we must crop it at block_size
        idx_cond = idx if idx.size(1) <= self.config.block_size else idx[:, -self.config.block_size:]
        # forward the model to get the logits for the index in the sequence
        logits, _ = self(idx_cond)
        # pluck the logits at the final step and scale by desired temperature
        logits = logits[:, -1, :] / temperature
        # optionally crop the logits to only the top k options
        if top_k is not None:
            v, _ = torch.topk(logits, min(top_k, logits.size(-1)))
            logits[logits < v[:, [-1]]] = -float('Inf')
        # apply softmax to convert logits to (normalized) probabilities
        probs = F.softmax(logits, dim=-1)
        # sample from the distribution
        idx_next = torch.multinomial(probs, num_samples=1)
        # append sampled index to the running sequence and continue
        idx = torch.cat((idx, idx_next), dim=1)

    return idx
```

Akış

```
Başla: idx (B, T)
||
|| → max_new_tokens kez tekrar:
||   ||
||   || Context'i kırp (block_size kadar)
||   || Modeli çalıştır → logits (B, 1, V)
||   || Son pozisyon + temperature → (B, V)
||   || Top-k filtresi (varsa)
||   || Softmax → olasılıklar
||   || Multinomial örnekle → idx_next (B, 1)
||   || idx'e ekle → idx (B, T+1)
||
|| → Son idx'i döndür
```

Pratik: Shakespeare Modeli Eğitme

Adım 1: Veriyi hazırla

```
python data\shakespeare_char\prepare.py
```

Çıktı:

```
length of dataset in characters: 1,115,394
all the unique characters: !$%&' ,-.:;?ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz
vocab size: 65
train has 1,003,854 tokens
val has 111,540 tokens
```

Tokenizer: Her karakter bir token. GPT-2'nin 50257 token'lı BPE'sinden çok farklı, sadece 65 token (karakter seviyesinde).

Adım 2: CPU kontrolü

```
python -c "import torch; print('CUDA:', torch.cuda.is_available())"
```

Çıktı: `CUDA: False` → CPU'da küçük model eğiteceğiz.

Adım 3: Eğitim (CPU için küçültülmüş)

```
python train.py config\train_shakespeare_char.py --device=cpu --compile=False --eval_iters=20 --l
```

Model: 4 katman, 128 embedding, ~1M parametre (default'tan 10 kat küçük).

Eğitim sonu:

```
iter 1999: loss 1.9002, time 40.78ms
step 2000: train loss 1.7648, val loss 1.8857
saving checkpoint to out-shakespeare-char
iter 2000: loss 1.6958, time 534.48ms
```

Loss 4.2'den 1.77'ye düştü. Model öğrendi!

Adım 4: Örnekle

```
python sample.py --out_dir=out-shakespeare-char --device=cpu --num_samples=3 --max_new_tokens=500
```

Sonuçlar ve Değerlendirme

Modelin ÜRETTİĞİ örnek

ROMEO:

My a lord perfect en.

MENENIUS:

Yet read, not that latt alt a to duke,
And in womenry house hemon the course,
The is a fiend to shall Levoule quores'd onsel
That I comed a my kid not angead of treant
the fail the of coursier groam'dly
The a look him the proverbs you him the Prase.

KING HENRY VI:

Rill shirng Marry:

I fath is sider' yours ere in say dold it me fain,
when sto the dough, nake unry I like own thee
To sigre to blood, And lay took?

Modelin ÖĞRENDİĞİ şeyler

1. **Shakespeare oyun formatı:** Karakter ismi → iki nokta → diyalog
2. **Gerçek karakter isimleri:** ROMEO, MENENIUS, KING HENRY VI, COMINIUS — hepsi gerçek!
3. **Yazım kuralları:** Büyük harf, noktalama, cümle yapısı
4. **İngilizce kelimeler:** lord, perfect, down, say, read, house, course, look, blood
5. **Gramer ritmi:** Özne-yüklem, bağlaçlar

Öğrenemediği şeyler

1. **Anlam:** Reship, latt, Levoule → uydurma kelimeler
2. **Tutarlılık:** Cümleler birbirine bağlanmıyor
3. **Mantık:** Gramer var ama anlam yok

Neden bu önemli?

Karakter seviyesinde, sadece 1M parametrelili, 2000 adım eğitilen küçük bir model bile bu kadar yapı keşfediyorsa, sözcük seviyesinde, milyarlarca parametrelili, yüzlerce milyar tokenle eğitilen modeller (GPT-4, Claude, Llama) neden bu kadar etkileyici olduğu anlaşılıyor.

Sadece ölçek farkı. Aynı mimari.

Şey	Bu model	GPT-4
Parametre	~1M	~1.7T
Katman	4	~120
Embedding	128	~12,000+
Eğitim adımı	2,000	Milyarlarca
Veri	1 MB	Petabayt

Öğrenilen Kavramlar (Toplu Liste)

Mimari

- ✓ Token, embedding, position embedding
- ✓ Self-attention (Q, K, V)
- ✓ Multi-head attention
- ✓ Causal masking
- ✓ Scaled dot-product attention
- ✓ Softmax
- ✓ Feedforward (MLP) ve GELU
- ✓ Residual connection
- ✓ Pre-norm vs post-norm
- ✓ LayerNorm
- ✓ Weight tying

Eğitim

- ✓ Cross entropy loss
- ✓ Train/val split
- ✓ Overfitting kavramı
- ✓ Learning rate, batch size, gradient accumulation
- ✓ Dropout

Üretim

- ✓ Autoregressive generation
- ✓ Temperature sampling

- ☒ Top-k sampling
- ☒ Multinomial örnekleme
- ☒ Context window (block_size)

PyTorch

- ☒ nn.Module yapısı
- ☒ **init** vs forward
- ☒ nn.Parameter, register_buffer
- ☒ nn.Linear, nn.Embedding, nn.LayerNorm, nn.Dropout
- ☒ Tensor boyutları ve `.view()`, `.transpose()`
- ☒ Broadcasting

Kaynaklar

- [Karpathy / nanoGPT](#)
- [The Illustrated Transformer \(Jay Alammar\)](#)
- [Karpathy: Let's build GPT from scratch \(YouTube\)](#)
- [Attention is All You Need \(orijinal makale\)](#)

Sıradaki Adımlar

- Daha uzun/büyük eğitim
- Türkçe veri ile eğitim
- `train.py` 'ın incelenmesi (eğitim döngüsü)
- Llama'nın özel detayları: RoPE, RMSNorm, SwiGLU
- Fine-tuning (hazır modeli kendi veriyle eğitme)

Bu not, bir GitHub repo'sundan başlayıp kendi GPT modelini eğitme yolculuğunun kaydıdır. Mimariyi sıfırdan, kod satır satır anlayarak öğrenmek için referans olarak kullanılabilir.